End of Day Report

Name	Atibali Saiyed
Date	12 November 2026
Topic	JavaScript (ES6+) DOM · TypeScript Fundamentals · QA Automation Foundations & HTTP
Github	https://github.com/Atibali/Gateway-Training/tree/main/Week2/Day4/

Part 1 — JavaScript (ES6+) DOM Basics

Selecting elements — Learned to select elements using `getElementById`, `querySelector` (first match), and `querySelectorAll` (all matches using a CSS selector). Also noted the difference between static NodeLists and live HTMLCollections.

Handling events — Used `addEventListener` instead of inline event handlers because it supports multiple handlers and removal. Learned about the event object, event bubbling, and event delegation, where one parent listener handles events from multiple child elements, including dynamically added elements.

Updating the page — Learned to update `textContent` (safer for plain text) versus `innerHTML` (which can introduce XSS risks when handling untrusted input), set attributes, manage classes with `classList`, create, insert, and remove elements, and use `DocumentFragment` for efficient bulk insertion.

Part 1 — TypeScript Fundamentals

Set-up and configuration — Understood that TypeScript provides static type checking to catch many errors before runtime and that its types are erased during compilation. Practised `npm install typescript`, `tsc --init`, and key `tsconfig.json` options such as `target`, `strict`, `outDir`, and `rootDir`. Learned to enable `strict` from the beginning.

Basic types — Covered `string`, `number`, `boolean`, arrays (`number[]` / `Array<string>`), and tuples (fixed-length arrays with position-specific types). Learned that `any` disables type checking, while `unknown` requires type narrowing before use. Also practised type inference, where TypeScript derives a type from an initial value.

Shaping data — Used interfaces and type aliases, optional (`?`) and `readonly` properties, union types (`string | number`), and literal types (`'north' | 'south'`). Understood that interfaces are useful for extendable object shapes, while type aliases support more flexible type definitions.

Functions — Typed function parameters and return values, optional parameters (placed after required parameters), default parameters, `void` and `never` return types, and typed arrow functions.

Classes — Applied access modifiers (`public`, `private`, `protected`, `readonly`), parameter properties shorthand, `implements` for interface contracts, and abstract classes (which cannot be instantiated directly and can require subclasses to implement members). Connected these concepts to Week 1 OOP principles: encapsulation, abstraction, inheritance, and polymorphism.

More TypeScript features — Covered numeric and string enums (and union literals as a lighter alternative), generics for reusable functions and types that preserve type relationships, type narrowing using `typeof`, `instanceof`, `in`, and custom `v is T` type guards, plus utility types such as `Partial`, `Pick`, and `Omit`.

Part 2— QA Automation Foundations & HTTP

JavaScript and TypeScript for automation — Focused on practical language features used in automation: `async/await` (awaiting each required step to avoid flaky tests), sequential `await` versus `Promise.all` for independent operations, and `for...of` instead of `forEach` for asynchronous iteration. Also reviewed objects (destructuring, shallow spread copies, and `structuredClone` for isolation), array methods (`filter`, `map`, `find`, `some`, `every`), table-driven tests, modules (named, default, and `import type`), and classes for Page Object and API client patterns.

HTTP methods and idempotency — Learned the purpose of GET, POST, PUT, PATCH, and DELETE. Understood idempotency and why retrying a non-idempotent POST request may create duplicate data.

HTTP status codes — Covered status-code groups and important distinctions for testers, including 401 (unauthenticated) versus 403 (forbidden), and 400 (bad request) versus 422 (unprocessable content). Learned to assert the exact expected status code where appropriate.

Headers and JSON — Reviewed request and response headers such as `Content-Type`, `Authorization`, `Location`, and `Set-Cookie` security flags. Learned that `JSON.stringify` omits `undefined` object properties, `res.json()` is asynchronous, and response bodies can be validated using matchers such as `toMatchObject`.

Inspecting API traffic — Practised using the browser Network tab to understand real API calls: record activity first, filter Fetch/XHR requests, inspect request headers, payloads, and responses, copy requests as cURL, and apply network throttling to help expose timing and race-condition issues.

Summary

Today I covered three connected areas: JavaScript DOM manipulation, the TypeScript type system, and how both support QA automation through async/await, classes (page objects and API clients), and HTTP fundamentals for understanding real API traffic.